

LTLf synthesis under external knowledge

Prototype, early results

Nadav Alon

August 11, 2026

What is this

A **prototype** that runs five synthesis methods under one interface, and a prototype benchmark suite.

- Five methods that produce controllers from a goal formula, a variable partition, and a knowledge transducer.
- A parametric benchmark suite, raced against `ltlfsynt` where the comparison is legitimate.

The tools

The Methods

Every method takes a goal formula φ , a variable partition (I_f, I_k, O_f, O_k) , and two knowledge transducers $\mathcal{T}_{in}$, $\mathcal{T}_{out}$.

Method	Representation	Built on
DfaProduct	explicit DFA	Spot
NfaProduct	explicit NFA	MONA
MtdfaProduct	symbolic (MTDFA)	Spot
MtnfaProduct	symbolic (MTNFA)	MONA
OtfMtdfaProduct	symbolic, on-the-fly	Spot

Two binaries

ltlf-ek-synth - the synthesizer

```
--dfa-product | --nfa-product | --mtdfa-product | --mtnfa-product | --otf-mtdfa-product
--formula=STRING      the goal formula
--part-file PATH       the variable partition (or --inputs/--outputs shorthand)
--known-input-transducer T_in  --known-output-transducer T_out
--realizable           verdict only          --model-check  verify a controller
--benchmark PATH       per-stage timing report as JSON
```

ltlf-ek-deps - Finds dependent variables. Very tentative.

```
--formula=STRING      --direction in|out
--transducer PATH      write the resulting T_in / T_out
--emit-part PATH       write the updated part file
```

Using it

Example

The partition — a part file:

```
input_free:  a
input_known: k
output_free: o
output_known:
```

The goal: $\varphi = X[!] \quad k$

“at the next step, k holds” - k is a known variable,
realizability depends entirely on the known input
transducer $\mathcal{T}_{in}$.

The knowledge — $\mathcal{T}_{in}$ as HOA plus a
labelling:

```
States: 2    AP: 2 "a" "k"
--BODY--
State: 0
    [t] 1
State: 1
    [t] 1
--END--
%%LAMBDA
state 0: !k
state 1: k
```

In this case its realizable

Example

Just changing the known input transducer $\mathcal{T}_{in}$ makes the same formula unrealizable:

$\mathcal{T}_{in} = \text{tin-eventual-k}$: k turns on at step 1 and stays on.

```
$ ltlf-ek-synth --dfa-product \  
  --formula="X[!] k" \  
  --part-file part-a-k-o.txt \  
  --known-input-transducer tin-eventual-k.hoa
```

```
HOA: v1  
States: 2  
...  
exit code 0          (a controller)
```

$\mathcal{T}_{in} = \text{tin-delay-k}$: k tracks the environment's a , one step behind.

```
$ ltlf-ek-synth --dfa-product \  
  --formula="X[!] k" \  
  --part-file part-a-k-o.txt \  
  --known-input-transducer tin-delay-k.hoa
```

```
UNREALIZABLE  
exit code 20
```


Model Checking

The output is a controller that can be model-checked against the goal formula + knowledge transducers.

```
HOA: v1
States: 2
Start: 0
AP: 2 "a" "o"
acc-name: all
properties: trans-labels explicit-labels state-acc complete deterministic
spot-state-player: 0 1
controllable-AP: 1
--BODY--
State: 0      [t] 1
State: 1      [t] 0
--END--

$ ltlf-ek-synth --dfa-product --formula="X[!] k" --part-file part-a-k-o.txt \
  --known-input-transducer tin-eventual-k.hoa --model-check
SAFE
```

Tentative dependency generation

A dependency finding tool. Tentative because it is a quick prototype that was hacked together with AI, not the final artifact.

```
$ ltlf-ek-deps --formula="F(a ^ b)" --inputs a,b --outputs o --direction in
dependent inputs: a    (of a, b)
```

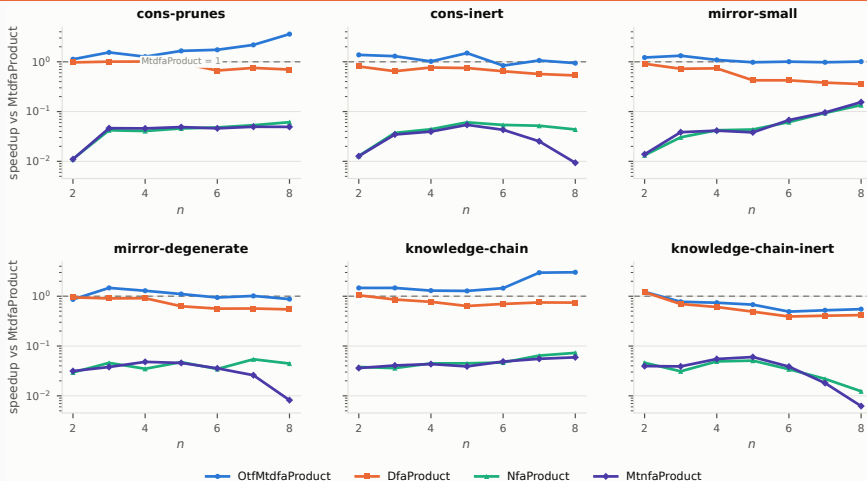
```
$ ltlf-ek-deps --formula="F(a ^ b)" --inputs a,b --outputs o --direction out
dependent outputs: none
```

Early results

Six parametric families, each scaling in n , none with deep thought behind them:

- `cons-prunes` - instances that try to benefit from the consistency filter
- `cons-inert` - instances that do not benefit from the consistency filter
- `mirror-small` - instances that (supposably) benefit from the nfa method
- `mirror-degenerate` - instances that (supposably) are hard for nfa and dfa both
- `knowledge-chain`, `knowledge-chain-inert` - instances where the *knowledge* grows with n , not just the formula

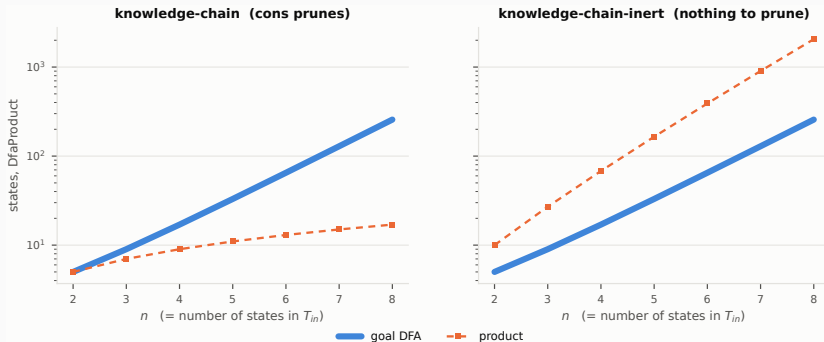
Speedup relative to MtdfaProduct



Mtdfa is baseline. Only the on-the-fly method beats it: 3–4 $\times$ where *cons* prunes (twice, independently), roughly free where it does not, **below 1** once the knowledge is large.

How big is the knowledge?

All four families above pin $\mathcal{T}_{in}$ at **one state**, so the product can never be bigger than the goal — an artifact of how they were written, not a property of the product. Here $|\mathcal{T}_{in}| = n$.



Where *cons* can prune, the product still collapses; where it cannot, it is exactly $n \times$ the goal — and that is where the on-the-fly method is **worst** ($0.55 \times$).